

프로젝트명	연관검색어 서비스 개발
프로젝트기간	2022.09~2022.12 (4개월)
참여인력	기획자 1, 개발자 1
사용한 기술스택	Python, Elasticsearch, Oracle, BERT, BoW
프로젝트 소개 *	
해당 프로젝트는 하나투어에서 연관검색어 서비스를 파일로 기획자가 시즌에 따라 고정적으로 값을 임의적으로 변경하고 있는 것을, 검색로그를 기반으로 다양한 연관된 키워드들이 동적으로 노출이 됨으로 써, 사용자의 검색 편의와 다양성을 제공한다. 하나투어에서의 연관검색어는 “추천검색어”와 “연관검색어”의 사전적 의미를 혼합한 형태이며, 검색로그와 매칭되는 상품데이터를 분석하여 “연관”관계를 맺는다. 검색로그 이외에, 도시검색어에 대해서는 태그정보를 상위에 노출될 수 있도록 한다. 이때 태그정보는 모든 태그에 대해 빈도수에 따라 상위에 노출된다. 어드민페이지에서 화이트/블랙 리스트를 통해 수동 “추천”과, 오타 및, 불필요한 데이터 처리를 제외하는 기능을 추가로 관리할 수 있도록 한다.	
전체 시스템구성 *	
1. 동의어, 유저 사전 동기화 2. 검색로그, 상품 데이터 수집 3. 데이터 정제 (전처리, 분석, 후처리) 4. 검색로그 데이터 그룹핑 5. 검색로그, 상품 데이터 할당 6. 연관검색어 데이터 학습 7. Elasticsearch 색인 포트폴리오의 “draw.io”를 참고	
프로젝트에 기여한 내용 *	
[모델 설계] 프로젝트의 올바른 방향에 대해서 여러 테스트를 수행한 후에, 하나투어에 맞는 최적의 방안을 구성 [개발] ● 예외 발생시, 빠르게 서비스에 적용이 될 수 있도록 Step 별로 기동이 될 수 있도록 구성 ● 패키지 모듈의 사전 경로, 배치 실행 설정을 프로젝트에 맞게 일부 수정	
프로젝트 성과 *	
● 연관검색어 서비스가 자동화가 됨에 있어서, 수동으로 배포하는 단계가 없어지고, 데이터가 다양하게 노출 ● 특정시즌에 대해 1~2달전부터 “예약”이라는 절차로 진행하는 하나투어의 특성에	

- 맞게, 사용자에게 필요한 검색어를 제공
- 수동으로 관리되어야 하는 키워드들에 대해서는 어드민에서 자동으로 처리

트러블 슈팅 경험 *

[word2vec, fasttext 사용이슈]

Train 단계에서 지역정보는 Word2Vec, 상품정보는 FastText 모델을 사용하여 학습을 진행하고자 했다.

하지만 학습이후 테스트 단계에서 간헐적으로 몇몇 키워드에서 전혀 관련이 없는 유사키워드가 도출되고 있었다.

(예: 일본 입력시 중남미 관련 키워드 노출)

[원인]

지역정보의 경우 중복되는 값을 제거한다면 500건 안쪽의 단어 개수가 만들어지는데, 이때 단어를 벡터화 할때 거리가 가까워서 (단어가 많고 학습량이 많아질수록 벡터거리가 멀어져서 뚜렷하게 구분이 됨) 엉뚱한 단어가 범위에 걸려서 노출하게 됨

[해결]

단순 빈도로만 연관관계를 체크하는 BoW 알고리즘을 사용

—

[Nori 사전 경로 이슈]

[원인]

PyNori는 패키지를 pip로 설치해야 하는데, 이때 로컬의 경로와, 배포되는 서버의 경로 (dev, qa, prod)는 다르게 된다.

[해결]

site-package의 경로가 아닌, 프로젝트 내부에 저장되어 있는 사전으로 형태소분석을 하기 위해 관련 py 파일들을 로컬로 옮겨 코드를 사전 파라미터를 받아서 처리하는 방식으로 변경하여 프로젝트 내의 사전파일들이 형태소분석 대상이 되도록 한다.

—

[Nori - KoreanAnalyzer synonym 이슈]

동의어 사전을 읽을 때, '=>'의 경우를 제대로 처리하지 못하는 이슈

[원인]

Java의 경우, 해쉬값과 Boolean형태의 Switching 변수로 컨트롤 하는것에 반해, Python에서는 Trie 구조로 되어있어서 이전의 값을 가지지 못함 (중복 X) 또한, 애초에 '=>'를 파싱하는 코드가 존재하지 않는다. (Java의 경우, Lucene 라이브러리 안에 SolrSynonymParser.java 존재)

【해결】

사전을 읽어 ' $\Rightarrow$ '를 가지는 라인을 가장 후순위로 뺀 이후, ' $\Rightarrow$ ' -> ','로 치환하여 저장한다.
그러면 ' $\Rightarrow$ '에 관련된 토큰값이 먼저 적용되고 이후에 ' $\Rightarrow$ '에 관한 단방향성이 적용되므로
같은 효과를 낸다.